

Aum Sri Sai Ram

Generalizable Facial Expression Recognition

Presented by:
Sai Narendra

Supervised By:

Prof. SVSN Sarma, Asst. Professor, Mathematics & Computer Science, SSSIHL

Dr. Darshan Gera, Asst. Professor, Mathematics & Computer Science, SSSIHL

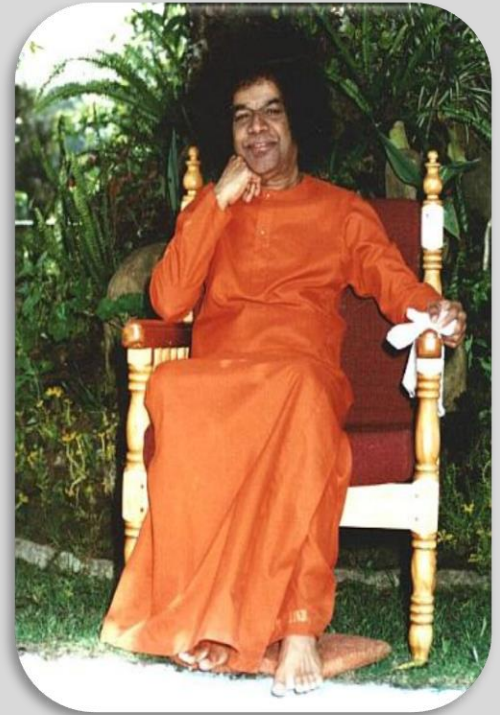
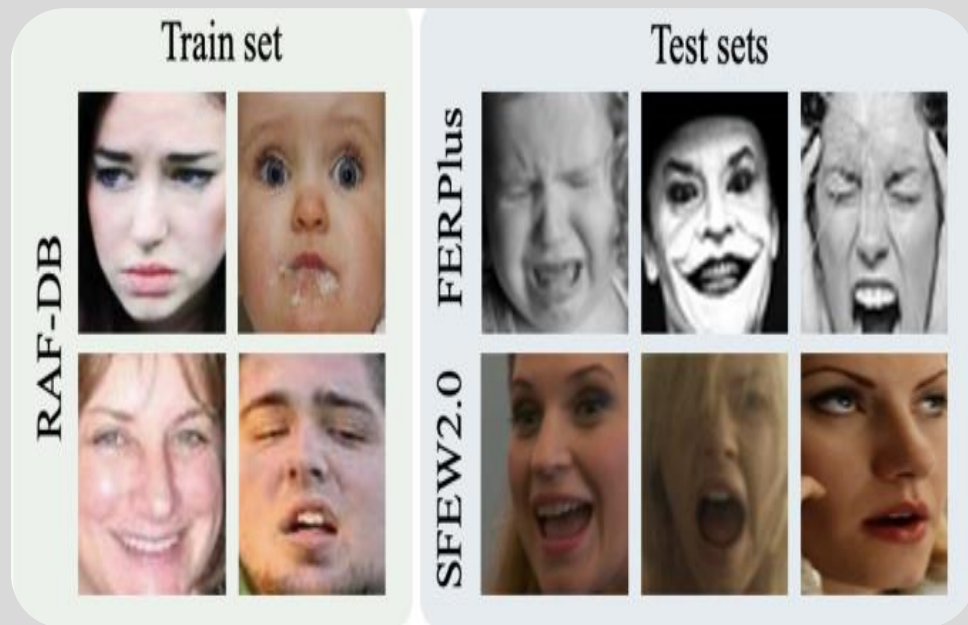


Table of contents:

1. Problem Statement
2. Introduction
3. Literature View
4. Methodology
5. Implementation Details
6. Results
7. Conclusion
8. Future Work

Problem statement:

- Facial Expression Recognition (FER) models struggle to generalize across different datasets due to domain gaps. (e.g., different lighting, ethnicities, camera qualities, or facial features).
- Existing domain adaptation techniques require labelled or unlabelled target domain samples for fine-tuning, which is often impractical.
- **Zero-Shot Generalization** → The goal is to **train on one dataset and generalize well to unseen test sets** without additional adaptation.

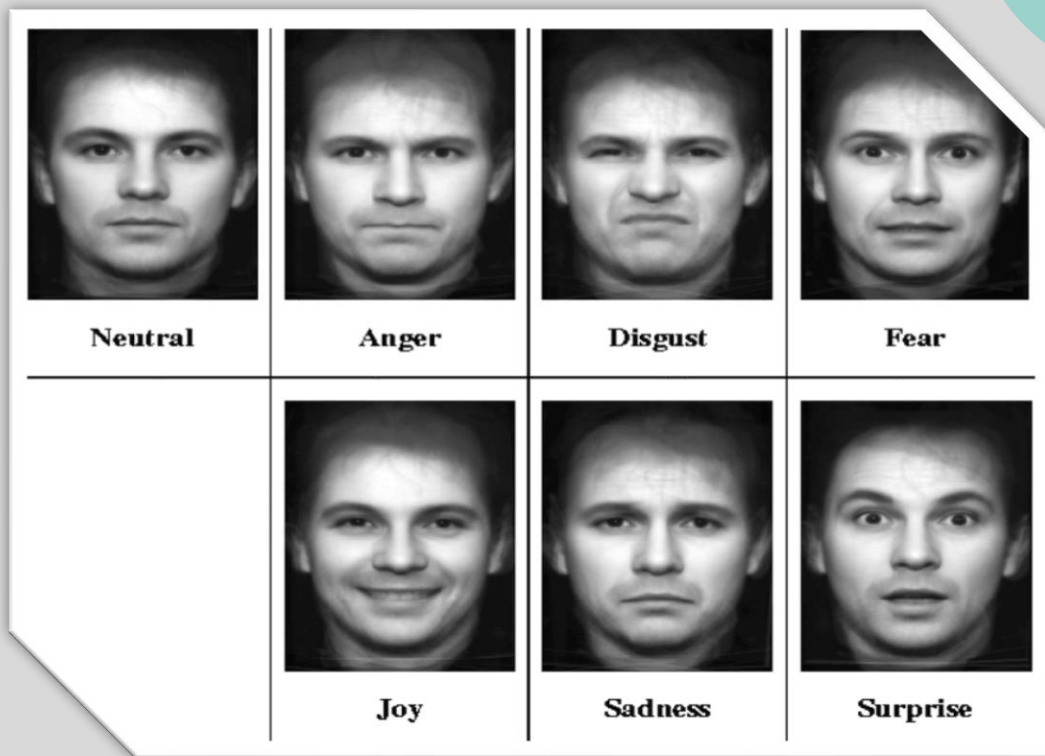


Referenced Paper :

Zhang, Yuhang, et al. "Generalizable Facial Expression Recognition." European Conference on Computer Vision. Cham: Springer Nature Switzerland, 2024.

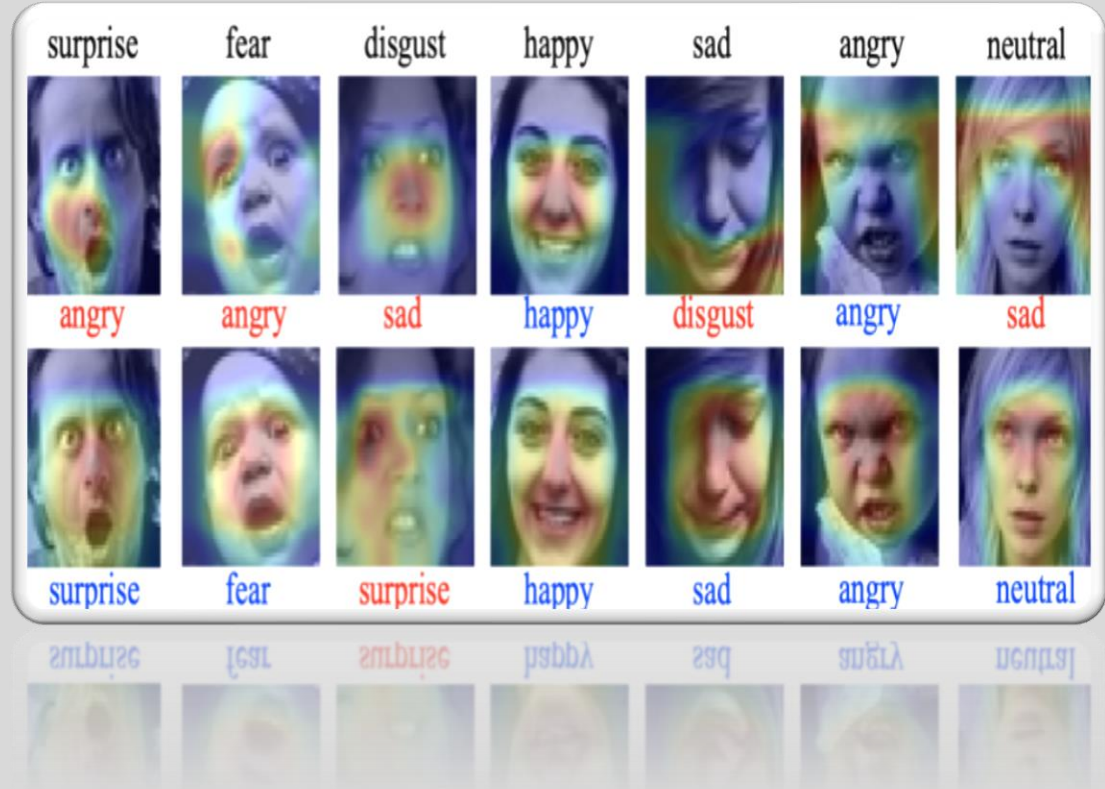
Introduction:

- Facial expression recognition plays a critical role in human-computer interaction.
- Despite the success of deep learning-based FER models, they exhibit poor generalization when tested on unseen datasets with domain shifts.
- Current domain adaptation approaches assume access to target domain data, which is not feasible in real-world scenarios.



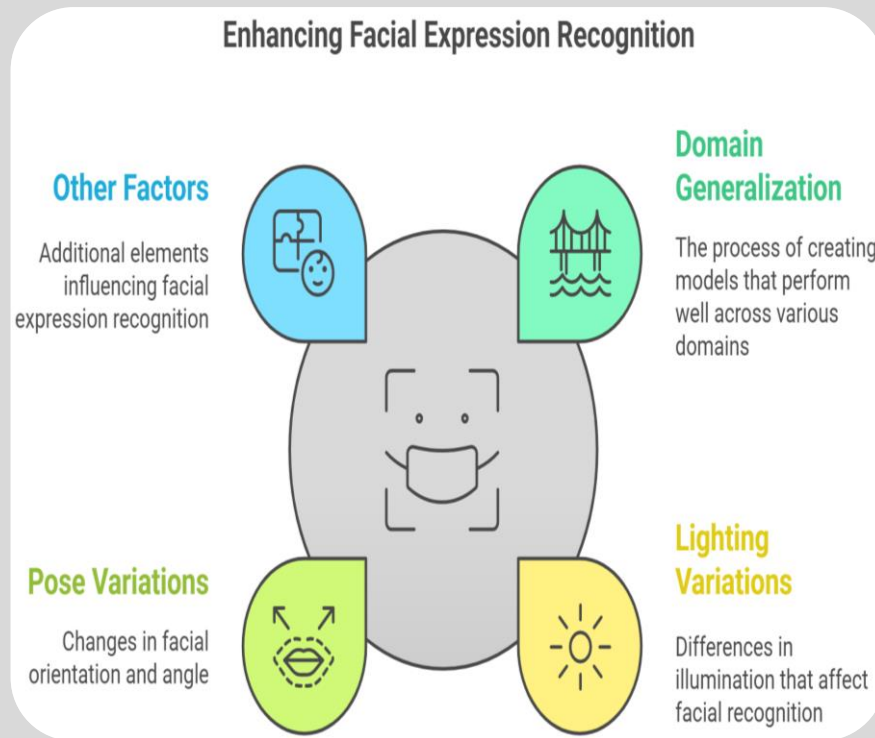
Key Challenges in FER:

- Domain Shift Problem
- Data Dependency
- Zero-Shot Generalization



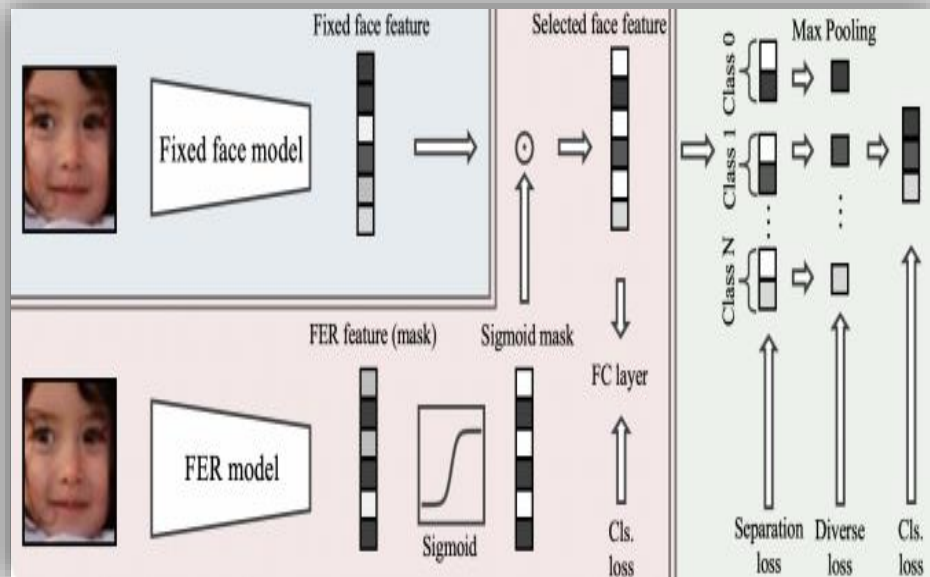
Literature View:

- Facial Expression Recognition (FER)
- Domain Generalization (DG)
- Pre-trained Feature Extractors



Methodology:

- Detect Faces First, Then Extract Expression Features
- Feature Extraction : Uses a fixed pre-trained model (**CLIP**) to extract face features, ensuring domain generalization.
- Mask Learning : Sigmoid Mask Learning on CLIP Features
- Channel Separation : Avoids fully connected layers by dividing features into seven expression classes.
- Channel-Diverse Loss for Better Feature Separation



Implementation :

- Model Architecture :
Backbone Model: ResNet-18
- Feature Extraction Model: CLIP VIT-B/32 (Vision Transformer)
- Training Configuration & Hyperparameters
Optimizer : Adam
Weight decay = 0.0001
- Learning Rate & Scheduler :
Initial Learning Rate (η) = 0.0002
Scheduler: ExponentialLR : Gamma = 0.9
- Loss Function Weights :
Channel-wise loss weight = 1.5
Diversity loss weight = 5

● Framework:

Step 1:

The input image is augmented by adding Gaussian noise and applying horizontal flipping to enhance the model's robustness.

Step 2:

The image is passed through the CLIP model, which uses a Vision Transformer (ViT) to extract high-level semantic features. These features are stored in a vector referred to as x .

Step 3:

In parallel, the same image is also passed through a ResNet-18 model up to its second-last layer (typically the pooling layer). The output is then flattened to prepare it for further processing.

Step 4:

A sigmoid activation is applied to the ResNet output to generate a soft mask. This mask is then applied element-wise to the CLIP features by multiplying the two vectors.

Step 5:

Finally, the masked CLIP features are passed through a fully connected (FC) layer to produce the final classification output.

This approach effectively combines the localized visual features from ResNet and the semantic-rich features from CLIP to make accurate predictions.

Losses used :

Classification Loss (Cls. loss):

Compares the predicted class with the true label

Usually done using crossentropyloss

Seperation loss:

Calculated by comparing features across different classes in the batch.

Encourages features from **different classes to be more distinct**.

Helps reduce overlap in feature space.

Diverse Loss:

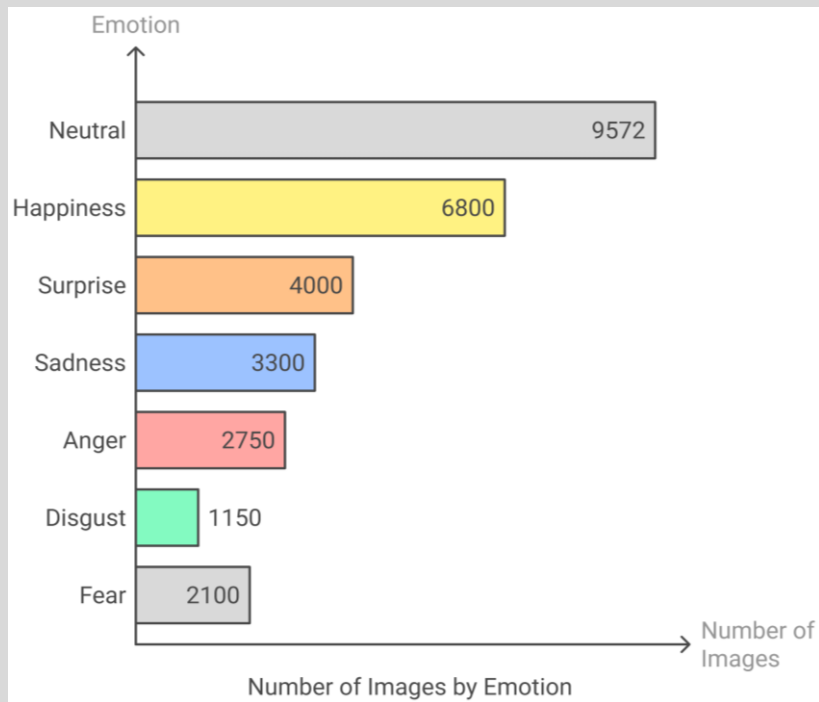
Encourages different neurons (dimensions) to capture **different aspects** of the data.

Promotes **feature diversity**, so the model doesn't overly rely on just a few dimensions.

At last all loses are added and used to perform backpropogation and update the trainable parameters.

Datasets :

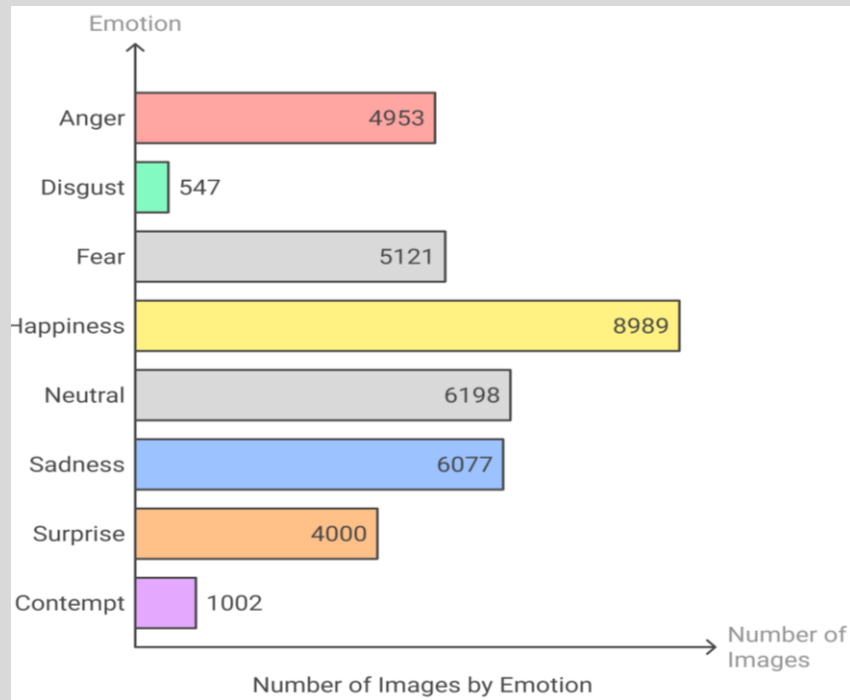
- ❖ Li, Shan, and Weihong Deng. "Reliable crowdsourcing and deep locality-preserving learning for unconstrained facial expression recognition." *IEEE Transactions on Image Processing*, vol. 28, no. 1, 2019, pp. 356–370.



I used FERPlus dataset for training and tested on RAF-DB and FERPlus datasets.



I used RAF-DB Dataset for training and tested on RAF-DB and FERPlus datasets.



Results:

Training Dataset	Testing Dataset	Accuracy	Our Accuracy
RAF-DB	RAF-DB	88.72	84.29
FERPlus	FERPlus	89.51	86.83
RAF-DB	FERPlus	73.16	86.20
FERPlus	RAF-DB	72.91	85.04

Performance of our method under different backbones:

Backbone	RAF-DB	FERPlus	Affect Net	SFEW2.0	MMA	Mean
MobileNet	84.65	61.33	43.45	43.21	40.67	54.66
MobileNet+our	85.07	64.97	42.91	45.45	43.11	56.30
Resnet-18	88.40	58.05	43.25	42.76	42.61	55.01
Resnet-18+our	88.72	73.16	45.86	52.86	56.80	63.48
Resnet-50	88.49	70.13	47.46	49.94	48.90	60.98
Resnet-50+our	89.05	75.26	47.49	51.07	57.46	64.07



Conclusion:

- I have implemented a generalizable FER method that significantly improves zero-shot performance across different datasets.
- By leveraging pre-trained models and selective feature masking, the model avoids overfitting and adapts better to unseen domains.



Future Work:

- Exploring different pre-trained models for better feature extraction.
Dino, ArgFace, GGFace
- Extending the approach to multi-label or compound expressions.
- Investigating more robust loss functions to further improve generalization.
Contrastive loss

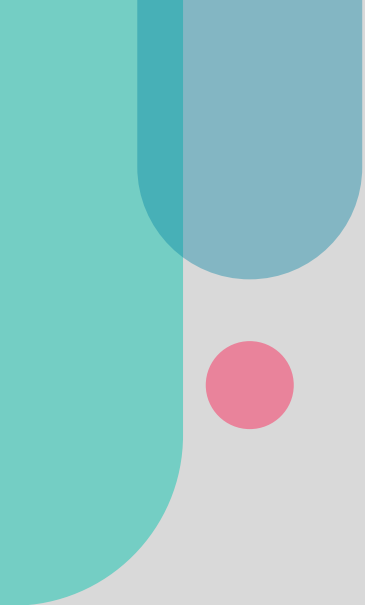
Acknowledgements:

Sri SVSN . Sarma

Assistant Professor, DMACS, Nandigiri Campus

Sri Darshan Gera

Assistant Professor, DMACS, Prashanthi Nilayam Campus, SSSIHL



References:

1. Zhang, Yuhang, et al. "Generalizable Facial Expression Recognition." European Conference on Computer Vision. Cham: Springer Nature Switzerland, 2024.
2. PyTorch Tutorials. Custom Dataset and DataLoaders Tutorial
3. "https://pytorch.org/tutorials/beginner/data_loading_tutorial.html".
4. Medium Article "<https://medium.com/analytics-vidhya/creating-a-custom-dataset-and-dataloader-in-pytorch-76f210a1df5d>".
5. Coursera course "<https://www.coursera.org/learn/convolutional-neural-networks/home>".

Thank you!
Jai sairam

